

Линейные и уникальные типы

Контекст требует формализации

Напомним правила вывода:

$$\frac{}{\Gamma, \theta \vdash \theta} \quad \frac{\Gamma, \varphi \vdash \psi}{\Gamma \vdash \varphi \rightarrow \psi} \quad \frac{\Gamma \vdash \varphi \rightarrow \psi \quad \Gamma \vdash \varphi}{\Gamma \vdash \psi}$$

Что такое контекст?

1. Это множество? Ведь $\alpha, \alpha \rightarrow \beta = \alpha \rightarrow \beta, \alpha$

$$\frac{\frac{}{\alpha, \alpha \rightarrow \beta \vdash \alpha \rightarrow \beta} \quad \frac{}{\alpha \rightarrow \beta, \alpha \vdash \alpha}}{\alpha \rightarrow \beta, \alpha \vdash \beta}$$

2. Разве это множество? Ведь $\alpha, \alpha \neq \alpha$.

$$\frac{\frac{\frac{}{\alpha, \alpha \vdash \alpha}}{\alpha \vdash \alpha \rightarrow \alpha}}{\vdash \alpha \rightarrow \alpha \rightarrow \alpha}$$

Структурные правила

Перестановка:

$$\frac{\Xi, \Delta, \Sigma, H \vdash \varphi}{\Xi, \Sigma, \Delta, H \vdash \varphi}$$

Сжатие:

$$\frac{\Xi, \delta, \delta \vdash \varphi}{\Xi, \delta \vdash \varphi}$$

Ослабление:

$$\frac{\Xi \vdash \varphi}{\Xi, \delta \vdash \varphi}$$

Сжатие и ослабление предполагают содержательные действия.

$$\frac{\frac{y : \beta \vdash y : \beta}{y : \beta, x : \alpha \vdash y : \beta}}{y : \beta \vdash \lambda x^\alpha. y : \alpha \rightarrow \beta}$$

$$\frac{\frac{x : \alpha \vdash x : \alpha \quad x : \alpha \vdash x : \alpha}{x : \alpha, x : \alpha \vdash (x, x) : \alpha \times \alpha}}{x : \alpha \vdash (x, x) : \alpha \times \alpha}$$

Два варианта удаления конъюнкции

Вариант 1 (классический):

$$\frac{\Gamma \vdash \alpha \times \beta}{\Gamma \vdash \alpha} \quad \frac{\Gamma \vdash \alpha \times \beta}{\Gamma \vdash \beta}$$

Вариант 2 (альтернативный):

$$\frac{\Gamma \vdash \alpha \times \beta \quad \Delta, \alpha, \beta \vdash \varphi}{\Gamma, \Delta \vdash \varphi}$$

Варианты эквивалентны при наличии структурных правил (например, классический через альтернативный):

$$\frac{\Gamma \vdash \alpha \times \beta \quad \frac{\overline{\alpha \vdash \alpha}}{\alpha, \beta \vdash \alpha}}{\Gamma \vdash \alpha}$$

Изоморфизм Карри-Ховарда

- ▶ Сжатие — копирование значения
- ▶ Ослабление — удаление значения
- ▶ Классическая конъюнкция:

$$\frac{\Gamma \vdash P : \alpha \times \beta}{\Gamma \vdash \text{fst}(P) : \alpha} \quad \frac{\Gamma \vdash P : \alpha \times \beta}{\Gamma \vdash \text{snd}(P) : \beta}$$

- ▶ Альтернативная конъюнкция:

$$\frac{\Gamma \vdash P : \alpha \times \beta \quad \Delta, x : \alpha, y : \beta \vdash E : \varphi}{\Gamma, \Delta \vdash \text{case } P \text{ of } (x, y) \rightarrow E : \varphi}$$

Линейная интуиционистская логика (вариант Филиппа Вадлера)

Грамматика:

$$\varphi ::= X \mid \varphi \multimap \varphi \mid \varphi \otimes \varphi \mid \varphi \& \varphi \mid \varphi \oplus \varphi \mid \mid !\varphi$$

Связки носят специальные названия: линейная импликация, мультипликативная и аддитивная конъюнкция, аддитивная дизъюнкция, фабрика («точно», «конечно»).

В линейной классической логике ещё рассматривают связки $\varphi \wp \varphi$ и $?\varphi$, в изоморфизме Карри-Ховарда они не используются.

Два типа контекстов: $\langle \alpha \rangle$ — линейный, $[\beta]$ — интуиционистский

Структурные правила:

$$\frac{\Xi, \Gamma, \Delta, H \vdash \alpha}{\Xi, \Delta, \Gamma, H \vdash \alpha} \quad \frac{\Gamma, [\alpha], [\alpha] \vdash \beta}{\Gamma, [\alpha] \vdash \beta} \quad \frac{\Gamma \vdash \beta}{\Gamma, [\alpha] \vdash \beta}$$

Аксиомы:

$$\overline{[\alpha] \vdash \alpha} \quad \overline{\langle \alpha \rangle \vdash \alpha}$$

Правила для связок

Правила для «конечно» (возможно тиражировать построение α):

$$\frac{[\Gamma] \vdash \alpha}{[\Gamma] \vdash !\alpha} \quad \frac{\Gamma \vdash !\alpha \quad \Delta, [\alpha] \vdash \beta}{\Gamma, \Delta \vdash \beta}$$

Линейная импликация:

$$\frac{\Gamma, \langle \alpha \rangle \vdash \beta}{\Gamma \vdash \alpha \multimap \beta} \quad \frac{\Gamma \vdash \alpha \multimap \beta \quad \Delta \vdash \alpha}{\Gamma, \Delta \vdash \beta}$$

Правила для связок: конъюнкция и дизъюнкция

Мультипликативная конъюнкция (возможно построить и α и β одновременно):

$$\frac{\Gamma \vdash \alpha \quad \Delta \vdash \beta}{\Gamma, \Delta \vdash \alpha \otimes \beta} \quad \frac{\Gamma \vdash \alpha \otimes \beta \quad \Delta, \langle \alpha \rangle, \langle \beta \rangle \vdash \varphi}{\Gamma, \Delta \vdash \varphi}$$

Аддитивная конъюнкция (возможно построить α и возможно построить β , что-то одно по нашему выбору):

$$\frac{\Gamma \vdash \alpha \quad \Gamma \vdash \beta}{\Gamma \vdash \alpha \& \beta} \quad \frac{\Gamma \vdash \alpha \& \beta}{\Gamma \vdash \alpha} \quad \frac{\Gamma \vdash \alpha \& \beta}{\Gamma \vdash \beta}$$

Аддитивная дизъюнкция (возможно построить α или возможно построить β , что-то одно по их выбору):

$$\frac{\Gamma \vdash \alpha}{\Gamma \vdash \alpha \oplus \beta} \quad \frac{\Gamma \vdash \beta}{\Gamma \vdash \alpha \oplus \beta} \quad \frac{\Gamma \vdash \alpha \oplus \beta \quad \Delta, \langle \alpha \rangle \vdash \varphi \quad \Delta, \langle \beta \rangle \vdash \varphi}{\Gamma, \Delta \vdash \varphi}$$

Пример: интуиционистская импликация

Введём обозначение:

$$\alpha \rightarrow \beta := !\alpha \multimap \beta$$

Заметим, что такая импликация ведёт себя как интуиционистская:

$$\frac{\overline{\langle !\alpha \rangle \vdash !\alpha} \quad \Gamma, [\alpha] \vdash \beta}{\Gamma, \langle !\alpha \rangle \vdash \beta} \\ \hline \Gamma \vdash !\alpha \multimap \beta$$

$$\frac{\Gamma \vdash !\alpha \multimap \beta \quad \frac{[\Delta] \vdash \alpha}{[\Delta] \vdash !\alpha}}{\Gamma, [\Delta] \vdash \beta}$$

Вложение остальных интуиционистских связок в линейные

Например, можно так:

$$\alpha \rightarrow \beta := !\alpha \multimap \beta$$

$$\alpha \times \beta := \alpha \& \beta$$

$$\alpha + \beta := !\alpha \oplus !\beta$$

Комбинаторный базис BCKW

- ▶ $B := \lambda x. \lambda y. \lambda z. x (y z)$ (комбинатор Z , Zusammensetzung)
- ▶ $C := \lambda x. \lambda y. \lambda z. x z y$ (комбинатор T , verTauschnung)
- ▶ $K := \lambda x. \lambda y. x$ (Konstanz)
- ▶ $W := \lambda x. \lambda y. x y y$

BC — линейная логика (значение нельзя ни удалить, ни скопировать)

BCK — аффинная логика (значение можно удалить, но не скопировать)

$BCKW$ — интуиционистская логика

Почему? Например, $W : (\alpha \rightarrow \alpha \rightarrow \beta) \rightarrow (\alpha \rightarrow \beta)$.

Реализация Уникальные типы

Язык

Роды:

$$\kappa ::= \mathcal{T} \mid \mathcal{U} \mid \star \mid \kappa \rightarrow \kappa$$

Типы:

$$\begin{array}{ll} \text{Int} & : \mathcal{T} \\ \rightarrow & : \star \rightarrow \star \rightarrow \mathcal{T} \\ \circ, \times & : \mathcal{U} \\ \vee, \wedge & : \mathcal{U} \rightarrow \mathcal{U} \rightarrow \mathcal{U} \\ \neg & : \mathcal{U} \rightarrow \mathcal{U} \\ \text{Attr} & : \mathcal{T} \rightarrow \mathcal{U} \rightarrow \star \end{array}$$

Обозначения:

$$\begin{array}{l} t^u := \text{Attr } t \ u \\ a \rightarrow^u b := \text{Attr } (a \rightarrow b) \ u \end{array}$$

Значения:

$$E ::= x^{\odot} \mid x^{\otimes} \mid \lambda x. E \mid E \ E$$

Типизация

Правила типизации имеют такой вид:

$$\Gamma \vdash E : \tau|_{fv}$$

Здесь Γ сопоставляет переменным типы рода $\star$. fv сопоставляет переменным типы рода $\mathcal{U}$.

Правила вывода:

$$\overline{\Gamma, x : t^u \vdash x^{\odot} : t^u|_{x:u}}$$

$$\overline{\Gamma, x : t^{\times} \vdash x^{\otimes} : t^{\times}|_{x:\times}}$$

$$\frac{\Gamma, x : a \vdash E : b|_{fv}}{\Gamma \vdash \lambda x. E : a \rightarrow^{\vee^{fv'}} b|_{fv'}} \quad fv' := fv \setminus \{x : \tau\}$$

$$\frac{\Gamma \vdash E_1 : a \rightarrow^u b|_{fv_1} \quad \Gamma \vdash E_2 : a|_{fv_2}}{\Gamma \vdash E_1 E_2 : b|_{fv_1 \cup fv_2}}$$

Смысл булевский выражений

Будем считать уникальность истиной, а неуникальность — ложью. И рассмотрим, например, функцию `fst`:

$$\text{fst} : (t^u, s^v)^{w \vee u} \rightarrow t^u$$

Это то же самое, что и

$$\text{fst} : (t^u, s^v)^w \rightarrow t^u, w \leq u$$

Однако, подобные выражения могут быть разрешены с помощью *булевской унификации*.

Где применяются линейные/уникальные типы

Ручное распределение памяти:

```
void compute() {  
    char* x = new char[1024];  
    char* y = x;  
    y[10] = 'a';  
    delete [] x;          // delete y; нельзя! будут ошибки.  
}
```

Автоматическое распределение памяти (сборка мусора, подсчёт ссылок):

```
public void compute() {  
    int[] x = new int[1024];  
    int[] y = x;  
    x[10] = 15;  
}  
  
fn compute() {  
    let x = Arc::new("abcde");  
    let y = x.clone();  
}
```

А давайте посчитаем количество ссылок при компиляции. Значение линейного/аффинного типа всегда существует в единственном экземпляре.

Практический пример: Раст

- ▶ В Расте (если не использовать небезопасные конструкции) ссылки можно копировать неограниченно, ограничения в «заимствовании» (borrow):
 1. На чтение (immutable, неизменяющее заимствование): количество одновременных заимствований неограничено;
 2. На запись (mutable, изменяющее заимствование): только однократно.

```
fn main() {  
    let mut v = [1,2,3].to_vec();  
    let vv = &mut v;  
    if let Some(v_0) = v.get(0) {    // заимствование v для чтения  
        vv.insert(2, 123);          // заимствование v для записи  
        println!("v[0] = {}", v_0);  
    }  
}  
  
>>> error[E0502]: cannot borrow 'v' as immutable because it is  
        also borrowed as mutable
```

- ▶ Значение можно «удалить» (drop) — скорее похоже на аффинные типы.
- ▶ Правила вывода задаются неформально.

Линейные и аффинные типы: отличия

- ▶ С аффинными типами допустимо ослабление — нельзя (дёшево) размножать, но можно уничтожить (массив, вектор, хэш-таблица).
- ▶ Внешний мир для программы (монада IO) — значение линейного типа. Нельзя создать, удалить, можно только изменить. Изменение производится пользователем при вводе значения — или компьютером, при выводе.
- ▶ Однако, IO в Хаскеле только имитирует поведение линейного типа, поскольку монады можно удалять, копировать и склеивать с помощью bind:

$$\begin{array}{ll} \text{return} : x \rightarrow \tau(x) & \text{(создание монады)} \\ (>>=) : \tau(x) \rightarrow (x \rightarrow \tau(y)) \rightarrow \tau(y) & \text{(bind, склейка монад)} \end{array}$$

- ▶ Рассмотрим следующие смелые манипуляции с внешним миром (IO):

```
main :: IO ()
main = let m = return () in (m1, m2, m3) = (m, m, m) in
      (m1 >>= \u -> putStr "Hello, ") >> (m2 >>= \u -> putStr "IO!")
```

Объектно-ориентированное программирование и ТТ

Отношение «быть подтипом»

Определение

Будем говорить, что A — подтип B :

$$A <: B$$

если значения типа A — часть множества значений типа B .

Пример

- ▶ В Java $\text{String} <: \text{Object}$.
- ▶ В Java $\text{int} \nless \text{Integer}$.

Ко- и контравариантность

Определение взято из теории категорий (и упрощено):

Определение

Пусть заданы два упорядоченных множества, $\langle C, < \rangle$ и $\langle D, < \rangle$. Ковариантное (контравариантное) отображение — $f : C \rightarrow D$, что

- ▶ при $x < y$ всегда $f(x) < f(y)$ (ковариантное отображение);
- ▶ при $x < y$ всегда $f(y) < f(x)$ (контравариантное отображение).

Операции на типах

Пусть $g : \star \rightarrow \star$. Что можно сказать о его ко- или контравариантности?

- ▶ Функции ковариантны по результату: $g(\sigma) := \text{int} \rightarrow \sigma$.

Если $\alpha <: \beta$, $a : g(\alpha)$, $b : g(\beta)$ то $a(n) : \alpha$, $b(n) : \beta$, поэтому функцию b можно всегда заменить на a . Отсюда,

$$\alpha <: \beta \text{ влечёт } g(\alpha) <: g(\beta)$$

- ▶ Функции контравариантны по аргументу: $g(\sigma) := \sigma \rightarrow \text{int}$.

Если $\alpha <: \beta$, $a : g(\alpha)$, $b : g(\beta)$, и $x : \alpha$ то $a(x)$ и $b(x)$ законны. Отсюда,

$$\alpha <: \beta \text{ влечёт } g(\beta) <: g(\alpha)$$

- ▶ Массивы инвариантны: $g(\alpha) := \alpha[]$.

С одной стороны, $g(\alpha).set : \alpha \rightarrow ()$, с другой стороны, $g(\alpha).get : \text{int} \rightarrow \alpha$.

В Java массивы ковариантны, что приводит к проверке времени исполнения.

Структурная и именная эквивалентность

```
struct X { a: int, b: char } x;  
struct Y { a: int, b: char } y;
```

Структурная эквивалентность: $X \approx Y$

Именная эквивалентность: $X \not\approx Y$

Исчисление $F_{<}$:

Язык:

$$T ::= x \mid \lambda x^{\tau}. T \mid T \ T \mid \lambda \alpha <: \tau. T \mid t \ \tau$$

Типы:

$$\tau ::= \alpha \mid \top \mid \tau \rightarrow \tau \mid \forall \alpha <: \tau. \tau$$

Подтипизация:

$$\frac{}{\Gamma \vdash \sigma <: \sigma} \quad \frac{\Gamma \vdash \sigma <: \rho \quad \Gamma \vdash \rho <: \tau}{\Gamma \vdash \sigma <: \tau} \quad \frac{}{\Gamma \vdash \sigma <: \top}$$
$$\frac{}{\Gamma, \alpha <: \tau \vdash \alpha <: \tau} \quad \frac{\Gamma \vdash \tau_1 <: \sigma_1 \quad \Gamma \vdash \sigma_2 <: \tau_2}{\Gamma \vdash \sigma_1 \rightarrow \sigma_2 <: \tau_1 \rightarrow \tau_2}$$

Исчисление $F_{<}$:

Типизация: правила системы F со следующими добавлениями/изменениями

$$\frac{\Gamma, \alpha <: \tau_1 \vdash T_2 : \tau_2}{\Gamma \vdash \lambda \alpha <: \tau_1. T_2 : \forall \alpha <: \tau_1. \tau_2} \text{ тип. абстракция}$$

$$\frac{\Gamma \vdash T_1 : \forall \alpha <: \tau_{11}. \tau_{12} \quad \Gamma \vdash \tau_2 <: \tau_{11}}{\Gamma \vdash T_1 \tau_2 : \tau_{12}[\alpha := \tau_2]} \text{ тип. применение}$$

$$\frac{\Gamma \vdash T : \sigma \quad \Gamma \vdash \sigma <: \tau}{\Gamma \vdash T : \tau} \text{ приведение}$$

Полное и ядерное правила

Ядерное правило подтипизации:

$$\frac{\Gamma, \alpha <: \rho_1 \vdash \sigma_2 <: \tau_2}{\Gamma \vdash \forall \alpha <: \rho_1. \sigma_2 <: \forall \alpha <: \rho_1. \tau_2}$$

Полное правило подтипизации:

$$\frac{\Gamma \vdash \tau_1 <: \sigma_1 \quad \Gamma, \alpha <: \tau_1 \vdash \sigma_2 <: \tau_2}{\Gamma \vdash \forall \alpha <: \sigma_1. \sigma_2 <: \forall \alpha <: \tau_1. \tau_2}$$

Типизация для пар

Можно показать:

$$\frac{\Gamma \vdash \sigma_1 <: \tau_1 \quad \Gamma \vdash \sigma_2 <: \tau_2}{\sigma_1 \& \sigma_2 <: \tau_1 \& \tau_2}$$

Что такое класс/объект

- ▶ Определим отношение подтипизации на кортежах — это даст наследование.

$$\text{struct } x_1 : \tau_1, \dots, x_n : \tau_n \text{ end} ::= \langle x_1 : \tau_1, \langle \dots \langle x_n : \tau_n, T : \top \rangle \rangle \rangle$$

Упростим, убрав имена полей (не теряя общности):

$$\text{struct } \tau_1, \dots, \tau_n \text{ end} ::= \tau_1 \& \dots (\tau_n \& \top)$$

Тогда

$$\tau_n \& \tau_{n+1} <: \tau_n \& \top$$

Отсюда,

$$\text{struct } x_1 : \tau_1, \dots, x_n : \tau_n \text{ end} <: \text{struct } x_1 : \tau_1, \dots, x_{n-1} : \tau_{n-1} \text{ end}$$

- ▶ Приведения типов и т.п. получаются согласно общим правилам $F_{<:}$.
- ▶ По необходимости добавим экзистенциальные типы.
- ▶ При необходимости сделать именную эквивалентность из структурной можно, включив какие-нибудь токены в тип .

Objective Caml

- ▶ ML — Meta Language, Робин Милнер, 1970е.
- ▶ Standard ML, 1983.
- ▶ Category Abstract Machine Language (Caml), 1985.
- ▶ ZINC project (ZINC Is Not CAML): “Toplevels considered harmful”, Ксавье Леруа (Xavier Leroy), 1990.
- ▶ Objective Caml, 1996.

Объекты, классы, подтипы

- ▶ Объектно-ориентированность — набор различных конструкций, которые можно собирать по-разному.
- ▶ Объектно-ориентированность в Окамле собрана иначе, чем в Джаве или в Смолтоке.
- ▶ В Окамле есть две различных конструкции: модули (соответствуют абстрактным типам данных), и объекты и классы (предназначены для построения иерархии наследования).
- ▶ Отношение подтипирования определено независимо от модулей и классов, в том числе и на обычных типах.

Полиморфный вариантный тип

Традиционный алгебраический тип не допускает пересечение вариантов:

```
type abc = A | B | C;;  
type ac = A | C;;           (* ac.C <> abc.C *)
```

Однако, есть полиморфный вариантный тип:

```
type abc = ['A | 'B | 'C];;  
type ac = ['A | 'C ];;
```

Заметим, что $[> 'A|'C] :> [> 'A|'B|'C]$.

Автоматический вывод типов не может построить правильное подтипирование, но можно указать его явно:

```
let f a = 'A;;           (* f : 'a -> [> 'A ] *)  
let g = (f () : ac);;    (* ошибка: у типа f() нет тэга 'C *)  
let g = (f () : ['A] :> ac);; (* g : ac, явное приведение типа *)
```

Модули: ковариантный интерфейс

```
module type Writer = sig
  type +'b t
  val empty : unit -> 'b t
  val push : 'b t -> 'b -> 'b t
  val print_len : 'b t -> unit
end

module Writer : Writer = struct
  type 'b t = 'b list
  let empty () = []
  let push l x = x :: l
  let print_len l = print_int (List.length l)
end

let w = Writer.empty ();;
let w = Writer.push w 'A;;
let w = Writer.push w 'C;;
let u = (w : ['A|'C] Writer.t :> ['A|'C|'E] Writer.t);;
let u = Writer.push u 'E;;
```

Модули: контравариантный интерфейс

```
module type Counter = sig
  type 'b t
  val empty : ('b->int->int) -> 'b t
  val push : 'b t -> 'b -> 'b t
  val print_len : 'b t -> unit
end
```

```
module Counter : Counter = struct
  type 'b t = int * ('b->int->int)
  let empty f = (0,f)
  let push (l,f) x = (f x l, f)
  let print_len (l,f) = print_int l
end
```

```
let w = Counter.empty (fun x l -> l + match x with 'A -> 0 | 'B -> 1 | 'C -> 2);;
let w = Counter.push w 'A;;
let w = Counter.push w 'C;;
let u = (w : ['A|'C] Counter.t -> ['A] Counter.t);; (* однако, ['A] -> ['A|'C] *)
let u = Counter.push u 'A;;
```

Объекты

Зададим объект:

```
type square = < area : float; width : int >;;
```

```
let square w = object
  method area = Float.of_int (w * w)
  method width = w
end;;
```

И нечто общее:

```
let coin = object
  method shape = circle 5
  method color = "silver"
end;;
```

```
let map = object
  method shape = square 10
end;;
```

```
type item = < shape : shape >;
let items = [ (coin :> item) ; (map :> item) ];;
```

Формализация: подтипы

- ▶ “Быть подтипом” определяется рекурсивно — согласно структуре.
- ▶ Приведение типа:

$$\frac{\tau <: \tau' \quad \Gamma \vdash \alpha : \theta(\tau)}{\Gamma \vdash (\alpha : \tau <: \tau') : \theta(\tau')}$$

Формализация: экзистенциальные типы и модули

```
► module type Counter = sig
  type 'b t
  val empty : ('b->int->int) -> 'b t
  ...
end
```

То есть, $\exists \beta. E : (\beta \rightarrow N \rightarrow N) \rightarrow \tau(\beta)$

- Заменяем экзистенциальный тип (интерфейс модуля) на его каноническое представление: $(B \rightarrow N \rightarrow N) \rightarrow \tau(B)$
- Правила подтипирования для типа, на основании его канонического представления:

$$\frac{\Gamma, \alpha \vdash R <: R'[\alpha' := \tau]}{\Gamma \vdash \exists \alpha. R <: \exists \alpha'. R}$$